

TP Programmation réactive : Appels http

Faites les 10 premières étapes du site <https://www.rxjs-fruits.com/> pour assimiler la syntaxe reactive avant de commencer le TP.

Dans ce tp nous allons utiliser le service HttpClient d'Angular afin de récupérer notre liste de Todo depuis internet. N'oubliez pas de changer le fichier app.config.ts de votre application pour inclure la méthode provideHttpClient() dans vos providers comme ci dessous :

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(),
  ]
};
```

Important

La documentation de l'api se situe à la fin de ce document.

Il faut changer le type du champ "id" des todos pour le mettre de type string au lieu de number car l'api génère des id de type string

Récupération des todos

Nous allons modifier todo-service pour récupérer les todos depuis un serveur distant. il faut donc utiliser le service HttpClient dans notre service TodoService (grâce à l'injection de dépendance)

Votre méthode getTodos devra donc renvoyer un Observable<Todo[]>

- **getTodos() : Observable<Todo[]>**

Modifier aussi votre composant todo-list pour la souscription à l'observable

Ajout d'un nouveau todo

Modifier la méthode createTodo du service TodoService elle devra avoir la signature suivante :

- **createTodo(label: string) : Observable<boolean>**

Elle renvoie un observable de boolean avec true si la sauvegarde a été effectuée, false sinon (cas d'erreur).

Indice : vous devez utiliser la fonction catchError vu en cours pour renvoyer false

Modification d'un todo

Ajouter la méthode updateTodo au service TodoService, qui sera appelée lorsqu'un todo est mis à jour par l'utilisateur (soit quand le label est mis à jour, soit quand son statut est mis à jour). Elle devra avoir la signature suivante :

- **updateTodo(todo: Todo) : Observable<boolean>**

Elle renvoie un observable de boolean avec true si la sauvegarde a été effectuée, false sinon (cas d'erreur).

Indice : vous devez utiliser la fonction catchError vu en cours pour renvoyer false

Suppression d'un todo

Ajouter la méthode deleteTodo au service TodoService, qui sera appelée lorsqu'un todo est supprimé par l'utilisateur (on peut rajouter une croix à coté du crayon d'edition). Elle devra avoir la signature suivante :

- **deleteTodo(id: string) : Observable<boolean>**

Elle renvoie un observable de boolean avec true si la suppression a été effectuée, false sinon (cas d'erreur).

Indices :

-vous devez utiliser la fonction catchError vu en cours pour renvoyer false
-c'est dans le todo-list component qu'il faut appeler le todo service pour supprimer votre Todo. Donc faire un nouvel evenement dans todo-item qui sera émis quand l'utilisateur cliquera sur l'icone de suppression puis l'intercepter dans todo-list. (C'est la meme idée que l'évènement updatedTodo actuel)

Ajouter un spinner de chargement

Grace à matérialize, ajoutez un spinner de chargement (<https://materializecss.com/preloader.html>) le temps que l'appel réseau se fasse. Celui ci doit prendre toute la page et disparaître pour laisser place à la liste des todos quand celle-ci est chargée

Pour simuler la lenteur du réseau, vous pouvez ajouter ?delay=XXXX à l'url avec XXXX le nombre de millisecondes de latence. Par exemple pour ajouter 2 secondes de latence, l'url est la suivante :

<https://europe-west1-cours-angular-263913.cloudfunctions.net/todoapp/todo?delay=2000>

Documentation API:

Méthode: GET (récupérer les todos)

URL : <https://europe-west1-cours-angular-263913.cloudfunctions.net/todoapp/todo>

Type de données de réponse: TodoResponse

Méthode: DELETE (supprimer un todos)

URL : <https://europe-west1-cours-angular-263913.cloudfunctions.net/todoapp/todo/<ID DU TODO A SUPPRIMER>>

Méthode: POST (ajouter un todos)

URL : <https://europe-west1-cours-angular-263913.cloudfunctions.net/todoapp/todo>

Type de données de réponse: Todo

Body: CreateTodo

Méthode: Put (editer un todo)

URL : <https://europe-west1-cours-angular-263913.cloudfunctions.net/todoapp/todo/<ID DU TODO A EDITER>>

Type de données de réponse: Todo

Body: UpdateTodo

Modèle de données

```
interface TodoResponse {  
    todos : Todo[];  
}
```

```
interface CreateTodo {  
    label: string  
}
```

```
interface UpdateTodo {  
    label: string;  
    done: boolean  
}
```

```
interface Todo {  
    id: string;  
    label: string;  
    done: boolean;  
    creationDate: number;  
}
```